

Tutorial 11: Scouting

A practical local-server Screeps module

<docs/tutorials/11-scouting.md>

Episode 11: Scouting the Dark

"You reserved a room in Episode 8 without really looking at it first," your collaborator says. "That worked because it was empty. It won't always be – that's the same energy as signing a lease on a spot you drove past once, at night, with your high beams on."

You've been picking rooms by moving a creep in and eyeballing the client. That doesn't scale past one room, and it definitely doesn't scale past rooms you're considering but haven't committed to yet.

Goal

Turn "look at the map" into a system that remembers what it saw:

- Send a cheap scout creep to visit a list of candidate rooms
- Record what it finds into `Memory.rooms`, keyed by room name
- Use that intel to make an expansion decision without a creep standing there live
- Understand why the intel goes stale and how to tell when it has

Prerequisites

Tutorial 10 is complete:

- You've practiced the structured-prompt workflow and have at least one journal entry.
- The colony from Episode 8 is running remote mining in `Memory.remoteRoom`.

Step 1: List Candidate Rooms

You already know how to do this from Episode 8:

```
const exits = Game.map.describeExits(Game.spawns.Spawn1.room.name);
Memory.scoutTargets = Object.values(exits);
```

Check each candidate isn't closed before sending anything there:

```
Memory.scoutTargets.forEach((roomName) => {
  console.log(roomName, JSON.stringify(Game.map.getRoomStatus(roomName)));
});
```

`status` will read `"normal"` for anything you can freely enter. `"closed"` rooms should be dropped from `Memory.scoutTargets` – a creep sent there simply won't be able to enter.

Step 2: Add `role.scout.js`

A scout is the cheapest creep in this codebase – a single `MOVE` part, 50 energy, disposable:

```
const roleScout = {
  run(creep) {
    const targets = Memory.scoutTargets || [];
    if (targets.length === 0) {
      return;
    }
  }
}
```

```

    if (creep.memory.targetIndex === undefined) {
      creep.memory.targetIndex = 0;
    }

    const targetRoom = targets[creep.memory.targetIndex % targets.length];

    if (creep.room.name === targetRoom) {
      recordIntel(creep.room);
      creep.memory.targetIndex += 1;
      return;
    }

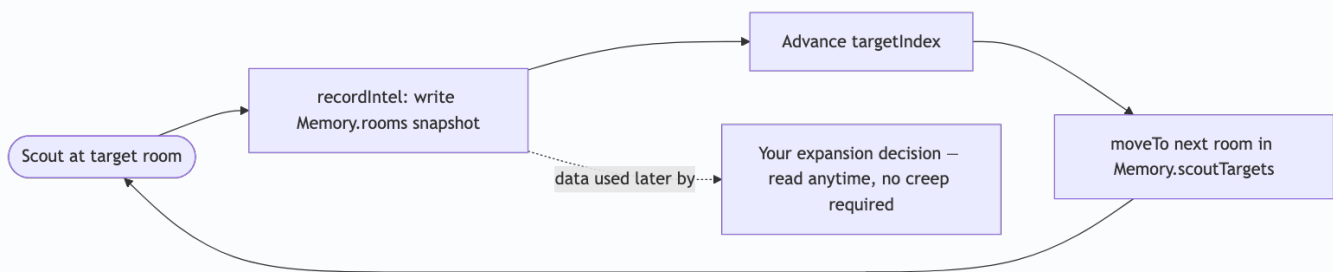
    creep.moveTo(new RoomPosition(25, 25, targetRoom), { visualizePathStyle: { stroke: '#00ffff' } });
  },
};

function recordIntel(room) {
  Memory.rooms = Memory.rooms || {};
  Memory.rooms[room.name] = {
    lastSeen: Game.time,
    owner: room.controller && room.controller.owner ? room.controller.owner.username : null,
    reserved: room.controller && room.controller.reservation ? room.controller.reservation.username :
  null,
    sources: room.find(FIND_SOURCES).length,
    hostileCount: room.find(FIND_HOSTILE_CREEPS).length,
  };
}

module.exports = roleScout;

```

The scout cycles through `Memory.scoutTargets` in order, recording a snapshot of each room the moment it arrives, then moving on to the next one. It never stops – a scout is meant to keep re-checking rooms, not just visit them once.



The loop and the decision are decoupled on purpose – the scout keeps circling regardless of whether you're actively reading `Memory.rooms` that tick.

Step 3: Wire It In

```

const roleScout = require('role.scout');

// add to ROLE_BODIES:
scout: [MOVE],

// add to POPULATION:
scout: 1,

// add to the dispatch loop:
} else if (creep.memory.role === 'scout') {
  roleScout.run(creep);
}

```

Checkpoint:

- A scout spawns and starts walking toward the first room in `Memory.scoutTargets`.
- `Memory.rooms` gains an entry the moment the scout enters that room.

Step 4: Read the Intel

```
Memory.rooms
```

Expected result: an object keyed by room name, each value holding `lastSeen`, `owner`, `reserved`, `sources`, and `hostileCount` – all without a creep currently standing in most of those rooms.

This is the point of scouting: the data outlives the creep's presence. You can make a decision about a room hours after your scout last visited it.

Step 4.5: Scout a Real Opponent, If One Exists

If an NPC bot is placed on this server (`docs/local-screeps.md` covers `bots.spawn('tooangel', '<room>')` or `'simplebot'`), add its room to `Memory.scoutTargets` too. `recordIntel` already captures `owner` – when your scout reaches that room, you'll see a real value there instead of `null`:

```
Memory.rooms['<bot-room-name>'].owner
```

Expected result: the bot's username (`tooangel` or `simplebot`), not empty. This is the difference between scouting theory and scouting practice – every other room in this episode is empty until proven otherwise; a bot's room is actually owned by something that's actively playing, the same way a real opponent's room would be.

 **Screenshot placeholder:** The map view zoomed out enough to show both your colony and the bot's claimed room, with the bot's owned tiles visibly color-coded differently – the moment "there's something else out here" stops being theoretical.

Step 5: Use Intel to Pick an Expansion Target

Instead of the manual "move a creep and look" process from Episode 8, query what you already know:

```
Object.entries(Memory.rooms)
  .filter(([name, info]) => !info.owner && !info.reserved && info.sources > 0 && info.hostileCount === 0)
  .map(([name, info]) => name)
```

Expected result: a list of room names that are unowned, unreserved, have at least one source, and had no hostiles the last time a scout looked.

Step 6: Understand Staleness

Every entry has a `lastSeen` tick. Intel doesn't update itself – it's a snapshot, not a live feed. A room that was empty 3,000 ticks ago might not be empty now.

```
Object.entries(Memory.rooms).map(([name, info]) => [name, Game.time - info.lastSeen])
```

Treat any entry with a large gap as "worth re-checking before committing a claimer," not as ground truth. A reasonable rule: if `Game.time - info.lastSeen` is larger than a full scouting loop's cycle time, send the scout back before you act on it.

Troubleshooting

If `Memory.rooms` never gets an entry for a target room, confirm the scout actually reached it: `Game.creeps['<scout-name>'].room.name` should eventually equal the target.

If the scout gets stuck at a room border, the destination coordinate `(25, 25)` may be unwalkable terrain in that specific room – Sereeps rooms aren't guaranteed to have open terrain at their exact center. Try nudging the target position, or let `moveTo` retry across a few ticks.

If `Game.map.getRoomStatus` throws or returns unexpected data for a room name, confirm the room name format matches what `describeExits` returned exactly – a hand-typed room name is an easy source of typos.

Completion Criteria

You are done when:

- A scout is patrolling `Memory.scoutTargets` on a loop.
- `Memory.rooms` has at least one entry with real, non-placeholder data.
- You can filter `Memory.rooms` down to viable expansion candidates using only console queries, no manual room visits.
- You understand what `LastSeen` is protecting you from.

Learning Notes

After completing the tutorial, write down:

- How many rooms are currently in `Memory.scoutTargets`, and how long does one full scouting loop take?
- What would happen to your Episode 8 remote-mining setup if `Memory.rooms[Memory.remoteRoom].hostileCount` suddenly became nonzero? Does anything in your code react to that yet?
- Why record `sources` count instead of the full source objects or IDs?
- What's the next piece of intel you'd want to track that isn't in `recordIntel` yet – mineral type, terrain quality, distance from home?
- If an NPC bot is running on this server, how far away is its room from yours? Would you want a scout watching it continuously, or is a one-time snapshot enough?

Next: Episode 12 – The First Skirmish

You now know what's out there. Some of it, eventually, will know about you too.

"Scouting tells you where the danger is before it arrives," your collaborator says. "It doesn't tell you what to do once it's here. That's combat, and none of these nine roles were built for it – great CRM, zero security team."

See `docs/roadmap.md` for the full season.